LangChain: A Powerful Framework for Language AI

What is LangChain?

LangChain is a framework that simplifies working with LLMs by providing a structured and modular way to build complex language AI applications.

- **Chain-Based Design:** Chains are modular units that orchestrate the interaction of LLMs, data sources, and other tools.
- **Integration with LLMs:** LangChain supports a wide range of LLMs, including OpenAI's GPT-3, Hugging Face models, and others.
- **Extensibility:** LangChain is designed for customization and can be extended to include custom models, tools, and data sources.

Why Use LangChain?

LangChain offers several advantages that make it a popular choice for developers working with LLMs.

- **Simplified Development:** LangChain provides a high-level abstraction, making it easier to build and maintain complex LLM-based applications.
- **Modular Design:** The modular architecture of LangChain allows you to easily combine different components and customize your application to suit specific needs.
- **Enhanced Capabilities:** LangChain offers features like memory, data augmentation, and tool integration, enabling you to create advanced language AI applications.

Other frameworks: LlamaIndex, FlowiseAI, CrewAI

Core Concepts and Architecture

LangChain is built around several core concepts that form the foundation of its architecture.

- **Chains:** Chains are the fundamental building blocks of LangChain applications. They define the flow of data and interactions between different components.
- **Models:** Models are the LLMs that provide the intelligence for LangChain applications. They can be used for text generation, translation, question answering, and more.
- **Prompts:** Prompts are the instructions you provide to the LLMs. They specify the task you want the model to perform.
- **Memory:** Memory allows chains to store and retrieve information from previous interactions, enabling more context-aware and personalized responses.
- **Agents:** Agents are autonomous entities that can perceive their environment, make decisions, and take actions to achieve their goals.
- **Document Loaders and Utils:** Document Loaders are responsible for ingesting various types of documents and converting them into a format that can be processed by LangChain. Utils are a collection of helpful functions and utilities that can be used across different parts of a LangChain application.

Setting Up LangChain

Getting started with LangChain is easy. Here's a step-by-step guide to install and set up the framework.

1. **Install LangChain**
   - Use pip to install the LangChain package: `pip install langchain`.
2. **Choose an LLM**
   - Select the LLM you want to use. Common options include OpenAI's GPT-3, Hugging Face models, or others.
3. **Configure API Keys**
   - Obtain API keys from your chosen LLM provider and store them securely in your environment.

Building Simple Chains

Let's start by building a basic LangChain chain that performs a simple text-based task.

1. **Define the Chain:** Create a chain that combines an LLM with a simple prompt.
2. **Provide Input:** Pass text input to the chain, which is then processed by the LLM.
3. **Output Results:** The chain outputs the processed text from the LLM.

CODE: Setup and Simple Chain → https://colab.research.google.com/drive/1MD6gL4b_MzKyznx7WJVcBgVnkKw29alD?usp=sharing

Advanced Use Cases

LangChain enables you to build advanced language AI applications by integrating with external data sources and tools.

Use Case	Description
Chatbots	Building conversational chatbots that leverage memory to provide context-aware responses.
Data Augmentation	Generating synthetic data to enhance training datasets for machine learning models.
Document Processing	Extracting insights from documents, summarizing text, or answering questions based on content.
Data Retrieval	Integrating with databases, APIs, and other data sources to provide relevant information to users.
Question Answering	Building systems that can answer questions by understanding context and retrieving relevant information.
Task Automation	Developing agents that can autonomously complete specific tasks using a variety of tools and data sources.

LangChain with Memory

LangChain provides memory capabilities to enhance the context awareness of your applications.

- **Chatbots**
  - Memory allows chatbots to remember past conversations and provide more relevant responses.
- **Interactive Assistants**
  - Memory enables interactive assistants to maintain a history of interactions and provide personalized support.
- **Storytelling**
  - Memory can be used to create narratives that evolve over time, remembering characters and events.

CODE: Complex Chain → https://colab.research.google.com/drive/1MbVmoHfcie14Q8l10xM3BDKzCMExD8TG?usp=sharing

Conclusion

LangChain is a powerful and versatile framework that empowers developers to build innovative and impactful language AI applications.

Future of Langchain

- **LangChain Expression Language (LCEL):** LCEL is a key part of LangChain, allowing you to build and organize chains of processes in a straightforward, declarative manner. It was designed to support taking prototypes directly into production without needing to alter any code. This means you can use LCEL to set up everything from basic "prompt + LLM" setups to intricate, multi-step workflows.

```
chain = prompt | model | parser
```

- **LangGraph:** LangGraph is a library for building stateful, multi-actor applications with LLMs, used to create agent and multi-agent workflows. Compared to other LLM frameworks, it offers these core benefits: cycles, controllability, and persistence. LangGraph allows you to define flows that involve cycles, essential for most agentic architectures, differentiating it from DAG-based solutions.

CODE: LCEL → https://colab.research.google.com/drive/1pcFXat2TX-u3EELipEeO_tGxY4AerrDl?usp=sharing

---

Q&A

Q1: Explain LangChain vs LangGraph in the simplest terms — what's each used for, the difference, and when to use which?

LangChain — the "building blocks" toolkit

Simple definition: LangChain is a toolbox of ready-made pieces (LLM connectors, prompts, memory, document loaders, vector-store connectors) that you **snap together in a straight line** to build an LLM app.

> 🥾 *Analogy: LangChain = a box of **LEGO pieces** + instructions to build something in **one straight path**: Step 1 → Step 2 → Step 3 → Done.*

Good for: simple, **linear** workflows — things that always go A → B → C, no looping back, no decisions mid-way.

LangGraph — the "flowchart with loops" engine

Simple definition: LangGraph lets you build workflows that can **loop, branch, pause, and retry** — not just a straight line, but a full flowchart/map with cycles.

> 🗺️ *Analogy: LangGraph = a **board game map** with branching paths — you can loop back to try again, take a different path based on a decision, or **pause and wait for a human** to make a move before continuing.*

Good for: AI **agents** — things that need to check their own work, retry if wrong, ask a human for approval mid-task, or have multiple agents talking to each other.

The core difference (one sentence each)

	LangChain	LangGraph
Shape	A straight **chain** (linear)	A **graph** with loops/branches (cyclical)
Best for	RAG pipelines, single-turn Q&A, quick prototypes	Agents that loop, retry, branch, pause for human input
Example	"Read doc → chunk it → embed → retrieve → answer" (one path, done)	"Try an answer → check it → if wrong, try again → if stuck, ask a human" (loops back)

When to use which (the simple rule)

- **Use LangChain** → if your app is a **straight pipeline** with no branching (RAG, simple chatbot, single Q&A).
- **Use LangGraph** → if your app needs to **loop, branch, recover from failure, or wait for a human** mid-process (real autonomous agents, multi-agent systems).

Important 2026 update — they're not really "either/or" anymore

As of their **joint 1.0 release (Oct 2025)**, LangChain and LangGraph work **together**:

- **LangChain** supplies the *pieces* (model connectors, tools, prompts)
- **LangGraph** is now the **actual engine underneath** that runs everything — even LangChain's own `create_agent()` runs *on* the LangGraph runtime
- The old `AgentExecutor` (LangChain's original agent runner) is now **deprecated**

So today: **LangChain = the parts, LangGraph = the engine that runs them.** For anything agent-like, you're using LangGraph whether you realize it or not.

One line: LangChain = building blocks for straight-line LLM pipelines (RAG, simple Q&A); LangGraph = the engine for looping, branching, agentic workflows (retry, human-in-the-loop, multi-agent) — and since 2025 they're merged in practice: LangChain gives you the pieces, LangGraph is the runtime that actually executes agents.

Sources:

- LangChain vs LangGraph: Key Differences Explained (2026)
- LangChain vs LangGraph: Performance, Cost & ROI (2026 Guide)
- LangChain vs LangGraph: Complete Comparison 2026

- LangChain vs LangGraph (2026): Which One Should You Build On?